

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 2

1 - Enunciado

Implemente duas versões da multiplicação de matriz por vetor ($M \times V$) em C: uma com acesso à matriz por linhas (laço interno variando a coluna) e outra por colunas (laço interno variando a linha). Meça o tempo de execução de cada versão usando uma função apropriada e execute testes com matrizes de diferentes tamanhos. Identifique a partir de que tamanho os tempos passam a divergir significativamente e explique por que isso ocorre, relacionando suas observações ao uso da memória cache e ao padrão de acesso à memória.

2 - Metodologia

Para analisar o desempenho durante o armazenamento de dados de forma que seja possível visualizar a diferença entre Row Major e Column Major foi feito a implementação em C de duas funções que realizam a mesma operação de multiplicação matriz-vetor, porém utilizando diferentes padrões de acesso à memória, foram realizadas múltiplas execuções para cada tamanho dessa forma reduzindo o ruído de medição causada pelo sistema operacional. Para medir a performance foi utilizado o walltime usando `clock_gettime`, que fornece uma medida do tempo real decorrido durante a execução. Os testes foram realizados com diferentes tamanhos de matrizes, variando de 32 até 2048 elementos por dimensão variando de 32 em 32 na ordem da matriz, foram realizadas múltiplas execuções para cada tamanho dessa forma reduzindo o ruído de medição causada pelo sistema operacional..

2.2 Algoritmo

Etapas do algoritmo:

1. Inicializar a matriz A, o vetor x e os vetores de resultado yl e yc.
2. Percorrer os diferentes tamanhos de matriz, de $N = 32$ até $N = 2048$ variando de 32 em 32.
3. Executar a multiplicação matriz-vetor por linhas (Row Major):
 - Para cada linha i, percorrer as colunas j;
 - Multiplicar $A[i][j]$ por $x[j]$;
 - Acumular o resultado em $y[i]$.
4. Executar a multiplicação matriz-vetor por colunas (Column Major):
 - Inicializar y com zero;
 - Para cada coluna j, percorrer as linhas i;
 - Multiplicar $A[i][j]$ por $x[j]$;
 - Acumular o resultado em $y[i]$.
5. Medir o tempo de execução de cada implementação utilizando `clock_gettime()`.
6. Calcular a razão entre os tempos de Column Major e Row Major para comparar o desempenho.
7. Registrar os resultados para cada tamanho de matriz e analisar a diferença de desempenho.

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB

Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

Tabela 1 - Configurações da máquina de teste

3 - Código Fonte

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

double *alocarMatriz(int n){
    double *m = malloc((size_t)n * n * sizeof(double));
    for(int i = 0; i < n*n; i++){
        m[i] = (double)(rand() % 100) / 7.0;
    }
    return m;
}

double *alocarVetor(int n){
    double *v = malloc((size_t)n * sizeof(double));
    for(int i = 0; i < n; i++){
        v[i] = (double)(rand() % 100) / 7.0;
    }
    return v;
}
```

```
void majorRow(double *A, double *x, double *y, int n){
```

```
    for(int i = 0; i < n; i++){
```

```
        double soma = 0.0;
```

```
        for(int j = 0; j < n; j++){
```

```
            soma += A[i * n + j] * x[j];
```

```
        }
```

```
        y[i] = soma;
```

```
    }
```

```
}
```

```
double medirTempo(void (*func)(double*, double*, double*, int), double *A, double *x, double  
*y, int n){
```

```
    struct timespec inicio, fim;
```

```
    clock_gettime(CLOCK_MONOTONIC, &inicio);
```

```
    func(A, x, y, n);
```

```
    clock_gettime(CLOCK_MONOTONIC, &fim);
```

```
    double tempo = (fim.tv_sec - inicio.tv_sec) + (fim.tv_nsec - inicio.tv_nsec) / 1e9;
```

```
    return tempo;
```

```
}
```

```
void majorColumn(double *A, double *x, double *y, int n){
```

```
    for(int i = 0; i < n; i++){
```

```
        y[i] = 0.0;
```

```
    }
```

```
    for(int j = 0; j < n; j++){
```

```
        for(int i = 0; i < n; i++){
```

```

    y[i] += A[i* n + j] * x[j];
}
}
}

int main(){
    int n_min = 32;
    int n_max = 2048;
    int passo = 32;

    FILE *arquivo = fopen("resultados.csv", "w");

    if (arquivo == NULL) {
        printf("Erro ao criar o arquivo CSV.\n");
        return 1;
    }

    printf("%-10s %-18s %-18s %-12s\n", "N", "Linhas (s)", "Colunas (s)", "Razao (Col/Lin)");
    printf("-----\n");

    for(int k = n_min; k <= n_max; k += passo){
        double *A = alocarMatriz(k);
        double *x = alocarVetor(k);
        double *yl = malloc(k * sizeof(double));
        double *yc = malloc(k * sizeof(double));

        int reps;
        if (k <= 512) {
            reps = 20;

```

```
} else if (k <= 1024) {  
    reps = 5;  
} else {  
    reps = 2;  
}  
  
double tLinhas = 1e18;  
double tColunas = 1e18;  
  
for (int m = 0; m < reps; m++){  
    double testRow = medirTempo(majorRow, A, x, yl, k);  
    if(testRow < tLinhas){  
        tLinhas = testRow;  
    }  
}  
  
for (int m = 0; m < reps; m++){  
    double testColumn = medirTempo(majorColumn, A, x, yc, k);  
    if(testColumn < tColunas){  
        tColunas = testColumn;  
    }  
}  
  
double razao = tColunas / tLinhas;  
  
// Mostra no terminal  
printf("%-10d %-18.6f %-18.6f %-12.2f\n", k, tLinhas, tColunas, razao);  
  
// Salva no CSV  
fprintf(arquivo, "%d,%.6f,%.6f,%.2f\n", k, tLinhas, tColunas, razao);
```

```
    free(A);  
    free(x);  
    free(yl);  
    free(yc);  
}  
  
// Fechar o arquivo  
fclose(arquivo);  
printf("\nResultados salvos em resultados.csv\n");  
  
return 0;  
}
```

4 - Resultados

Os testes foram aplicados para ambas as formas com passo de 32 , indo de $N=32$ até $N=2048$ sendo possível observar o comportamento de desempenho com o crescimento da matriz. Abaixo duas imagens com a saída após a execução os dados abaixo foram exportados para um arquivo resultados.csv que será usado para gerar os gráficos de comparação.

./main.py

N	Linhas (s)	Colunas (s)	Razao (Col/Lin)
32	0.000008	0.000010	1.21
64	0.000033	0.000038	1.17
96	0.000074	0.000086	1.16
128	0.000131	0.000135	1.03
160	0.000118	0.000134	1.14
192	0.000169	0.000192	1.14
224	0.000229	0.000261	1.14
256	0.000300	0.000365	1.22
288	0.000378	0.000429	1.14
320	0.000468	0.000528	1.13
352	0.000566	0.000637	1.13
384	0.000676	0.000759	1.12
416	0.000796	0.000894	1.12
448	0.000917	0.001053	1.15
480	0.001058	0.001283	1.21
512	0.001217	0.001853	1.52
544	0.001384	0.001851	1.34
576	0.001544	0.002049	1.33
608	0.001733	0.002119	1.22
640	0.001896	0.002369	1.25
672	0.002107	0.003034	1.44
704	0.002293	0.004009	1.75
736	0.002498	0.003648	1.46
768	0.002740	0.004167	1.52
800	0.002981	0.004460	1.50
832	0.003227	0.004841	1.50
864	0.003475	0.005253	1.51
896	0.003732	0.005605	1.50
928	0.004002	0.006150	1.54
960	0.004304	0.006512	1.51
992	0.004585	0.007124	1.55
1024	0.004886	0.021791	4.46

Figura 1: imagem do teste realizado

```
1056      0.005205      0.009302      1.79
1088      0.005548      0.010126      1.82
1120      0.005869      0.010746      1.83
1152      0.006208      0.012448      2.01
1184      0.006559      0.012533      1.91
1216      0.006881      0.013724      1.99
1248      0.007279      0.013678      1.88
1280      0.007683      0.018115      2.36
1312      0.008057      0.017305      2.15
1344      0.008462      0.017069      2.02
1376      0.008872      0.018877      2.13
1408      0.009281      0.017612      1.90
1440      0.009738      0.020401      2.09
1472      0.010108      0.020488      2.03
1504      0.010621      0.022740      2.14
1536      0.011023      0.053420      4.85
1568      0.011471      0.020863      1.82
1600      0.011875      0.019875      1.67
1632      0.012397      0.022488      1.81
1664      0.012957      0.021194      1.64
1696      0.013386      0.024324      1.82
1728      0.013956      0.024653      1.77
1760      0.014345      0.026644      1.86
1792      0.014960      0.047250      3.16
1824      0.015544      0.030208      1.94
1856      0.016052      0.029004      1.81
1888      0.016595      0.030782      1.85
1920      0.017165      0.030648      1.79
1952      0.017731      0.034515      1.95
1984      0.018333      0.033530      1.83
2016      0.018924      0.035172      1.86
2048      0.019531      0.107495      5.50

Resultados salvos em resultados.csv
~/Doc/g/Programacao_Paralela/Tarefa_2 on main !1 ?1
```

Figura 2: imagem do teste realizado

Como pode ser observado, os testes para matrizes pequenas (32x32 e 64x64) estão praticamente tendo o mesmo tempo.

Podemos observar essa variação através do gráfico, que foi gerado usando matplotlib.pyplot:

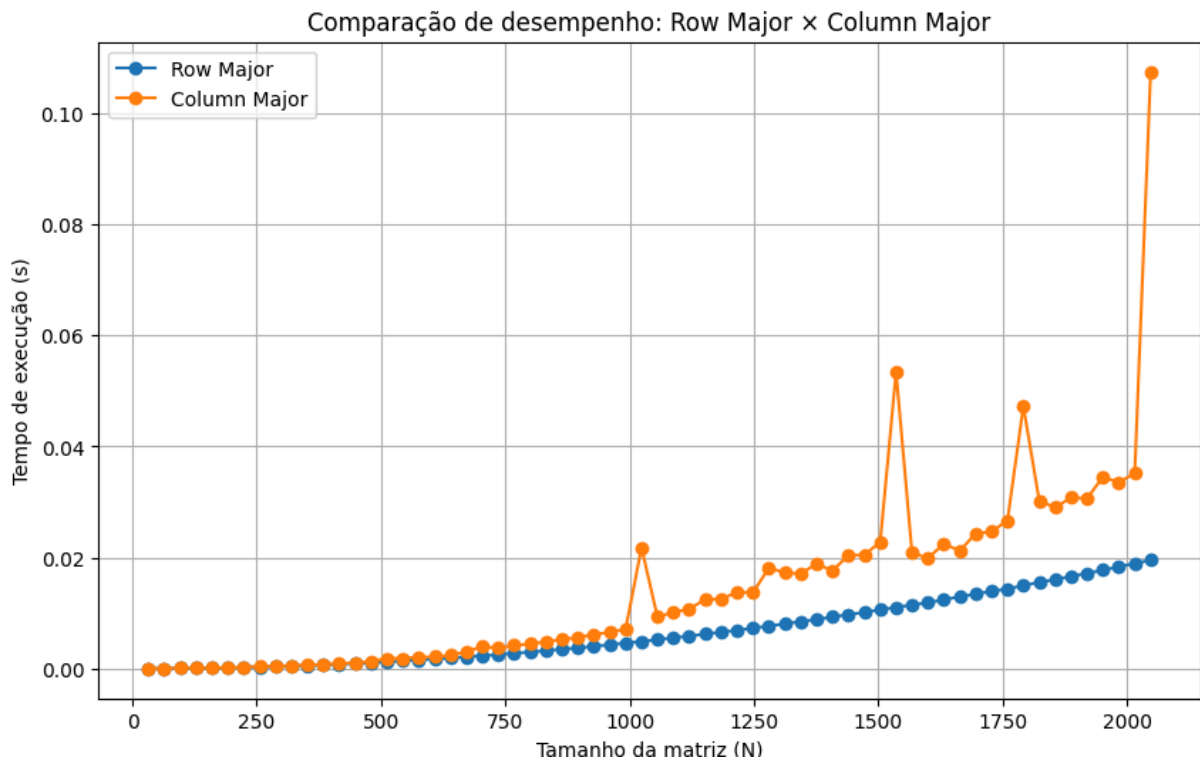


Figura 3: Tamanho x tempo

A figura 3 mostra que até $N=500$ as curvas têm um crescimento leve e permanecem quase iguais, porém à medida que esse número cresce a discrepância vai se tornando mais visível. Como em C a matriz é armazenada em forma linear, a versão por linha percorre a memória de forma contígua, logo cada linha da cache é trazida da memória junta, já na versão por colunas são aproveitados que não são descartados isso pode ser visualizado na figura abaixo..

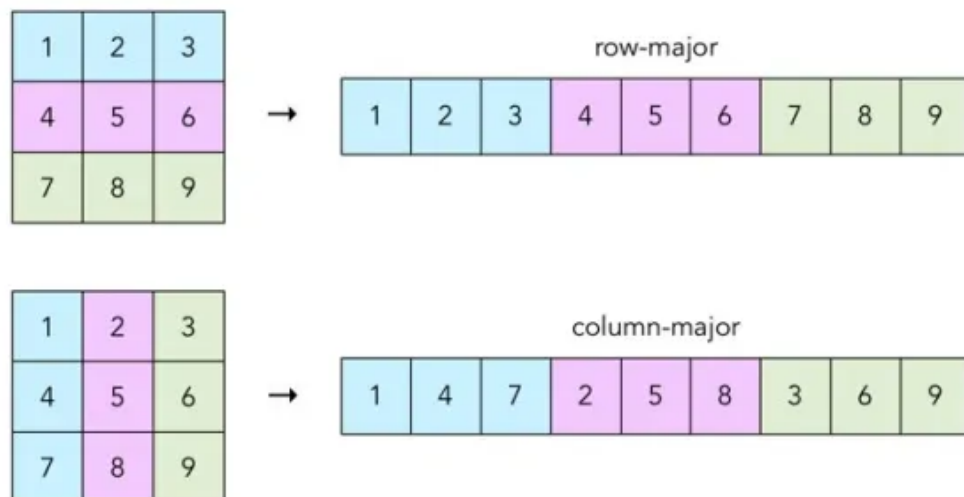


Figura 4: Armazenamentos na memória

Os picos isolados em $N=1024$, 1536 , 1792 e 2048 não são explicados só pelo tamanho total da matriz ultrapassar a cache mas são consistentes com conflitos de associatividade causado pelos

saltos gerados no armazenamento por colunas sendo necessário múltiplas linhas serem mapeadas para o mesmo conjunto de cache.

4 - Conclusão

O experimento coloca em prática conceitos vistos em aula como localidade temporal e espacial, além da forma como é armazenado um dado na cache e o quanto ter controle sobre esse acesso retorna como desempenho. Além disso, o experimento mostra que escolher a forma certa de percorrer e armazenar o dado pode ser crucial para projetos de alto desempenho.